

Media engineering

S M West and M T Norris

Modern computer networks are capable of presenting a user with huge amounts of information. If an end user is unable or unwilling to assimilate what is available then it is of little value. The key contention in this paper is that information has to be engineered, just like any other product, if it is to deliver to its potential. Presented here is a systematic approach to the design and build of information-rich systems that has been designated media engineering.

1. Introduction

The last few years of the 20th century have heralded the dawn of a new age in which access to information has become globally available (thanks to a powerful combination of computers and networks), and in which the provision of information is increasingly important. Furthermore, a significant change is afoot with many organisations now using computer-based information as a prime source of guidance and reference. The words and pictures on the screen are no longer simply background data — they are the main object of attention, the definitive copy [1].

Many of the traditional constraints on preparing and presenting information do not translate into a world where the computer is the focus of attention. No longer is text a sequence of statements — a linear story or argument supported by explanation, reason or illustration. More complex structures are now possible — it is standard practice to have computer-based documents that allow users to jump from one place to another, as they wish. The good news is that we have the prospect of vast amounts of infinitely malleable, readily changeable data along with an array of tools for moulding it as we please. Less welcome is the observation that the concepts and methods needed to turn a wealth of data into useful information are still in their infancy.

The intention in this paper is to present a systematic approach to preparing computer-based documents. This does not extend to the content as this can only be prepared or selected according to the provider's preference. Nor does it deal with the way in which information is presented — issues of artistic flair and good taste are quite separate from those of engineering. Our focus is on the application of a systematic and disciplined approach to the development, operation and maintenance of information products such as those associated with multimedia and the World Wide Web.

We start by outlining some of the reasons why new methods are needed in the information age. We then reflect on some of the principles that have served a related discipline, software engineering, well and show how they offer a sound basis for these new methods. Finally, we explain how the principles translate into techniques for designing and building information products — an approach we call 'media engineering'. Some of the practical aspects of media engineering are illustrated in a brief case study — more detail in this area can be found in the book from which this paper is abridged [2].

2. The need to engineer hypermedia

The first essential drive for media engineering comes from the use of dynamic, rather than sequential, presentation. The use of hypertext on computers means that the *non sequitur* and logical schism that every book author worries about can readily be removed by inserting a corrective link. Conversely, poorly placed links can leave the reader unable to infer any logical relationship between pieces of information. This might be acceptable to players of 'Dungeons and Dragons', but not to information professionals.

So, with such a basic change in capability, the implication is that the way in which information is designed needs to be re-examined. With new rules in the writing game, new techniques are called for.

Figure 1 illustrates this and shows a move from the user having to search an increasingly large information space under their own steam (usually referred to as surfing) to a situation where information is presented as a cogent resource.

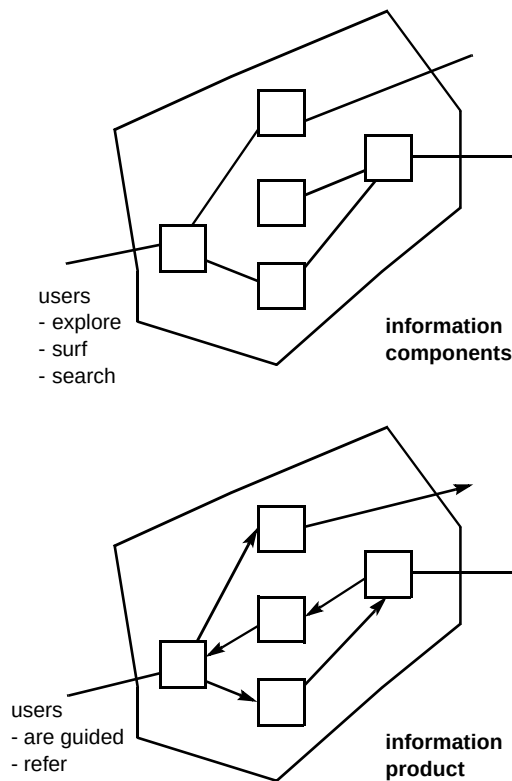


Fig 1 The move from surfing to guided tours.

It is not that the concepts of searching and surfing (known and loved throughout the Internet community) are going to disappear; more that the burgeoning demands on the information consumer will mean that they are complemented with information excursions where the viewer is guided through a subject. In the current model, it is up to the user to create their on-line map. In the media engineering model, the map starts to become part of the information product.

There is little guidance available on how to provide this level of support as yet. Some elements are in place. For instance, there are more than adequate guides on page design, production languages and delivery technology. The products for building and distributing information exist and are well documented. It is with the process for structuring the information that we are more concerned.

Our second defining statement on media engineering relates to its intended audience — who is it for. To answer this, we note that many of the barriers to providing information to a mass market disappear with computer-based information systems — it is no longer just the press that has the means to get their views across to a huge number of people. This allows a new equation to become reality: 'author = publisher'. The producer of a book, a guide or some publicity does not have to rely on a third party to make their work available. It is almost as easy for them to distribute it as it was to produce it in the first place. So an old order (rather like the privilege of reading and

writing, the preserve of the clergy in the middle ages) is broken and there are many more (potential) players in a previously bounded market-place.

The publishing revolution gave authors access to professional technicians who would undertake information presentation, replication and distribution on their behalf; the media revolution has placed the tools of presentation, replication and distribution directly in the hands of the authors.

3. Current problems

One thing that should be clarified at this point is why it is important to design information. After all, many people will have quite happily used the search engines available on the World Wide Web to locate items of interest. So, what are the problems? This section details some of the main ones.

3.1 On-line structure

Most people who have tried to find more than a very specific piece of information will feel that something better is needed when it comes to on-line structure. With the world so diverse and interlinked, few have the time or energy to build a complete picture for themselves. The user should have a clear and consistent view of:

- where they are,
- where they can go,
- how to get there quickly.

3.2 The global cottage industry

In a few short years, the variety and volume of accessible information has exploded and a 'global cottage industry' of Web page production has been created. Some of this has to be welcome. It is now relatively easy to track down a host of facts and references that would previously have taken several weeks and much shoe leather to locate. However, the quality, cogency and validity of what is on-line is by no means guaranteed.

3.3 Document translation

Commonly, the implementor of a multimedia system is confronted with a vast set of existing documentation produced by various packages (MS-Word, Pagemaker, WordPerfect, Powerpoint, etc). Tools are needed to convert these into hypermedia without having to rewrite or redraw large parts of them. In addition, decisions must be made with regard to new material — whether to use a special-

purpose multimedia authoring package or to use a conventional tool and perform a translation.

3.4 Configuration management

Current multimedia technology encourages the fragmenting of information into files that contain a few pages of text, a single picture, a single video sequence or a single sound. Even in quite a small multimedia production the number of individual files can grow to several hundred, and poor configuration management (or a complete absence of it) is common. Many Web site managers have encountered the problem of looking through the server file structure and finding a lot of redundant files, some which look like old versions of current pages while others look like test versions that have never been brought into service. However, it is impossible to tell for sure which ones are actually in use, so they cannot be deleted in case they are important. Likewise, users frequently experience error messages that tell them that something is amiss (e.g. '404 page not found'), but have no detail on why the problem has arisen nor what they can do about it.

3.5 Validation and testing

In a typical multimedia system, the number of 'links' (items of clickable text, buttons, icons, clickable pictures, etc) exceeds the number of files by a factor of about ten-to-one. So a system with a few hundred files is likely to have a few thousand links. It is important to test that all these links work, but testing several thousand links by hand is not feasible. Furthermore, it is difficult to be sure that a test is valid unless there is a documented design against which that test has been performed.

3.6 Navigation

One of the most important limitations of multimedia systems is navigation. It is all too easy to get 'lost in space' unless there is an underlying structure that lets the consumer of the information know with what sort of product they are dealing — and this structure has to be designed into the product from the outset.

3.7 Orientation

Anyone who has 'surfed' the World Wide Web will know that a few clicks of the mouse can take you into a maze of interconnected pages. So even after you have found out where you are, how do you find out where you want to go and how to get there?

3.8 Metrics and estimation

In a professional multimedia department, it is essential to be able to plan the job in terms of how long it will take, how many people are needed and how much it will cost to build (and maintain).

4. The analogy with software engineering

In addressing many of the problems listed above, it is apparent that they bear a striking resemblance to those that have dogged the software industry for over 25 years. Back in the mid-1960s, a new term was coined — 'software crisis' — which reflected the fact that the production of software-rich systems was running into the brick wall of complexity. The real cause of this problem was that software had, within a few short years, gone from a specialist activity to a potential solution for all technological problems. Everyone was doing it but there were no real norms or rules in place.

The initial response to the software crisis was to invent a new term — software engineering. The hope was that a systematic approach to software development would soothe the industry's ailment. The situation was not unlike one with which many of us are now familiar — lots of people solving their marketing, sales, advertising and operational ills by transferring their information on-line: the 'global cottage industry' that was mentioned earlier.

With software engineering, the industry developed some key concepts and methods which allowed the problem to be managed. For example, the notion of the software life cycle provided the view of development as a linked set of stages, each with a defined purpose of its own. Another key step was that of developing design notations, providing some means of being able to record and reason about abstract designs. Many other concepts (objects, use cases, naming and linking and binding) contributed to the development of software engineering.

It is the contention of the authors that the same general approaches can equally be applied to the problems outlined here and that a discipline of media engineering needs to be built up to guide the designer of complex information products, just as the discipline of software engineering evolved in response to increasingly complex software systems.

However, there is much that needs to be done before the whole situation becomes more systematic. We are still a way off being able to build information products of comparable size or with the same level of reproducibility of software products. The DIVA method was developed by the authors to begin to address the problem.

5. The DIVA method

It is not unreasonable to explore the solution that was eventually evolved for the software crisis — it might, if sufficiently abstracted from the details of software, be equally applicable to the media explosion of the 1990s. It was with this aim that the authors conceived the DIVA method.

DIVA provides a systematic approach which aims, if not actually to solve the problems that were set out earlier, at least to keep them under control. Although the techniques are relatively new to multimedia they are all based on various tried-and-tested software engineering approaches developed over the last two decades.

5.1 Basic ideas

Some of the main concepts of DIVA are given below.

- Discrimination

Many of the problems experienced in building multimedia systems relate to the interaction between content and structure. If the two become mixed up, there can be unwanted interactions — especially when (inevitable) changes are made.

The principle of discrimination requires that structure and content be separated. This may seem to deny a basic property of multimedia (namely that links can be inserted anywhere in text or pictures). However, in practice, such a separation can largely (though not completely) be maintained without compromising the inherent properties and advantages of multimedia. What is paramount is that the designer understands the distinction between content and structure, and applies appropriate levels of resource and attention to each.

- Encapsulation

This concept describes how elements of content are divided into small manageable items which are, as far as possible, independent of other elements. This division into small, independent units is obvious in a publication such as an encyclopaedia, where each article represents a discrete information object. It is less obvious in some other types of multimedia, but can be applied to good effect in almost all cases. The importance of encapsulation is that it allows:

— the clear identification of self-contained items that can be configuration managed, which opens the way for each information object to be edited, changed or enhanced without impact on the remainder of the multimedia system,

— the management of items of content quite separately from the structure of the information, thus supporting one of the other key concepts — that of discrimination; as far as possible, each information object contains either structure or content but not both.

- Information object

This is an identifiable multimedia item. In practice, it is likely to be a single disk file containing a self-contained set of information. This might consist of a few pages of text on a single subject, a single picture, a single video clip or a sound. Software engineering is concerned with the management of 'modules' of computer software; an information object is an analogous module of multimedia material. It is the smallest item that is managed — it must have a unique name and an identifiable place where it is stored; it must have attributes such as a 'creation date', a 'last modified date' and a 'version number'.

- Polycontiguity

It is a fundamental principle that there are multiple navigation paths through the same body of information. This principle is described as 'polycontiguity'. The author of a conventional book usually writes it on the basis that there is a single path through it (starting in the top left corner of the first page and running through linearly to the bottom, right corner of the last page). The richness of their story has to be folded into a sequential format. However, the author of a multimedia system must anticipate that people will follow complex threads of interconnection through the material. In order to match the cogency of a conventional book, the author must not only be aware that people will use multimedia in this way but must design the system with this in mind.

- Staples

Information objects need to be identified in some way in order to distinguish whether they are part of the content or part of the structure. It is also useful to have some way of identifying attributes not apparent to the end user. Examples of such attributes are the identification of some items as 'overview' material and others as 'reference' material.

In DIVA, information objects can be tagged (or stapled) with the set of attributes (or staples) that relate to them. This set of attributes needs to be defined and is dependent on the particular multimedia application.

As stated earlier on, DIVA draws heavily on the principles of software engineering. It will be clear to many in this field that the concepts of encapsulation and information objects have obvious counterparts in object-oriented software methods [3]. The full DIVA method embraces a wide range of software engineering concepts,

some theoretical, some pragmatic and some concerned with process.

This paper focuses on two of the particularly important principles of DIVA, again drawn from software engineering — the life cycle and the design notation.

5.2 Design notation

A notation is a way of writing down or drawing a design. This is something that is currently completely lacking from the field of media engineering. Most Web site designers could probably describe quite clearly the elements of the graphical design but would have some difficulty explaining the information design, including such items as the way that pages are linked together.

Indeed many sites have not been designed at all. The ‘designer’ has begun by linking a few pages together and then just expanded the site by a process of accretion — many sites display all the worst signs of having been created in this way. Even the best may only display consistency and usability because a principal designer has been able to hold the complete picture in their head while implementing the system. For most of us this is too great an intellectual challenge. Furthermore, we require some confidence that the site can continue to grow, evolve and be maintained even if the original designer moves on to other tasks. Without the architect’s plans for a house it is difficult to plan a new extension. To be able to write down or draw the design, we need a notation.

One of the innovations that software engineering brought to the software industry was the ability to describe and specify software designs in terms of specification languages, some textual, most of them pictorial. Many different notations for software have been devised, embodying varying degrees of formality and optimised to suit various areas of application. Although much of the early emphasis was on truly formal languages that had a strict mathematical basis (hence were quite analogous to the notations used, for example, in mechanical engineering), more long-term impact has probably been achieved by less rigorous, but more accessible, techniques such as entity-relationship notation, data flow diagrams, message sequence charts and object models [4].

When designing a media system, the major items that must be documented are the ‘pages’ of information and the ‘links’ between them. We have chosen to focus our notation on the links rather than the pages. The reason for this is that links are the things that allow users to browse and surf — they are also the main cause of runaway complexity.

When undertaking detailed page design, it could be claimed that there are many types of link. Media

engineering is concerned largely with designing at the system level rather than the page level, so needs to provide some means of controlling this underlying complexity. To achieve this, we propose just a few useful ways of categorising links at this level, thus helping to simplify the way we describe and document the design. Firstly there are two main categories of links which are referred to as transitive and constructional.

A **transitive** link is the kind that is visible to the user — it is activated by clicking on an element of hypertext or a button or some other object and, as a result, some change of context occurs, with a new page of information being presented to the user. This is the kind of link that everyone would think of as a link.

A **constructional** link is employed to construct a page from various objects — images, text, sound, and video. Languages such as HTML create composite pages by embedding links to the various media objects and although these links, when implemented as HTML, look much like transitive links, their function is quite different. In fact the user will not even be aware that these links exist — unless they fail to work.

In general, the system design concentrates on the transitive links, although the physical design will have to include all links.

Transitive links, in turn, consist of three major categories — local, unstructured and structured links.

- Local links are those whose scope is limited to a particular ‘page’, i.e. they are links to a named anchor within the same page — the design of these links can be left until late in the design process because they have no impact on the overall integrity of the system as a whole, an example being a World Wide Web page that begins with a contents list that links to subsequent sections lower down on the same page.
- Unstructured links take the user from a source location in the information space to a destination which has no structural relationship to the source, e.g. in printed documents, a footnote or a bibliography reference constitutes an unstructured link — if the reader is following a structured piece of information, following such a link takes them into a completely unrelated area from which their only choices are either to undertake a simple backtrack to their starting point or else to follow some completely new information thread.

An unstructured link is a thread that leads to an information cul-de-sac. It is rather like a ‘go to’ in a programming language, although it has a more auspicious place in the discipline of media engineering than the humble ‘go to’ has found in software engineering.

- Structured links are those between related pieces of information within a structure that is discernible to the user. Generally, following a link takes the user into an area from which they can return directly or via other pieces of related information, while retaining some context of 'where they are' in the information web, e.g. a menu bar that is common to a whole set of pages would be an implementation of structured links.

The distinction between structured and unstructured links is important and should be clearly made during the design stage. It is the structured links that should be the main focus of the design activity: they define the overall 'shape' of the multimedia system. They must be designed so that when following them, the user is provided with enough contextual information to understand where they are going and how they might find their way back. In general, the structured links must be tackled at an early stage of design and the unstructured links can be added at a later stage as they do not significantly impact the overall shape of the system.

When selecting a notation, our focus is on techniques that will allow us to document structured links. In our notation, we use the conventions shown in Fig 2.

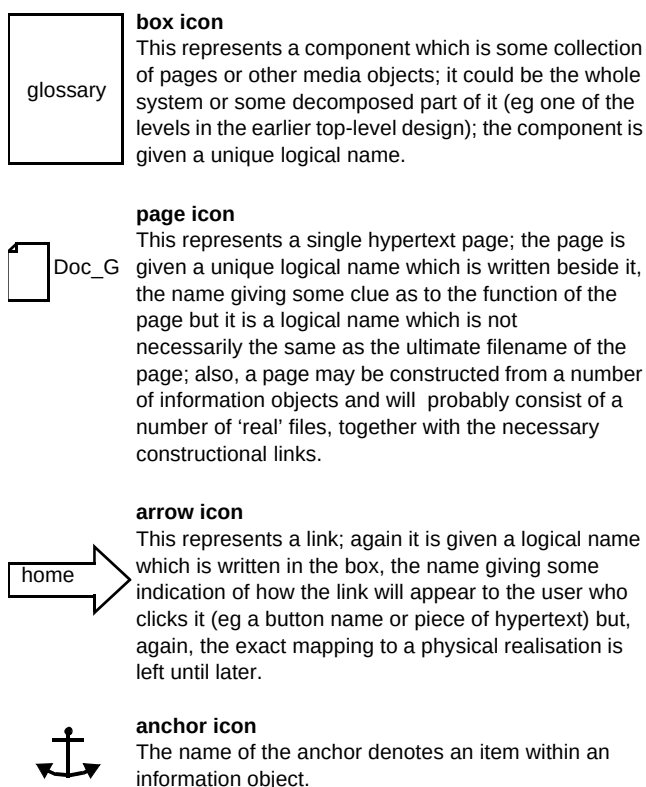


Fig 2 Notation conventions.

The way in which the notation has been used to specify an information product is illustrated in the next section. Before this, a little of the process side of media engineering is explained.

5.3 Media engineering life cycle

The life cycle recognises that complex undertakings require the whole end-to-end task — not just the creative part — to be brought under control. By considering the whole process from concept to maintenance and withdrawal, it is more likely that a systematic approach will be followed — one that defines what steps to take, what order to take them in, and the criteria for completing each stage. An explicit life cycle helps to structure design considerations and trade-offs between cost, quality and time-scale aspects of product development.

Figure 3 illustrates the proposed media engineering life cycle.

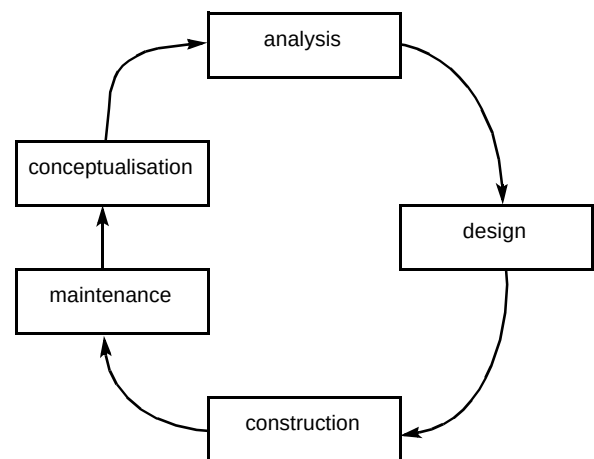


Fig 3 Media engineering life cycle.

Needless to say, this is based on a software engineering life cycle, one originally attributed to Booch [5].

During the stages, the following tasks are accomplished:

- Requirements
 - mission statement for the media system,
 - subject scope,
 - target audience,
 - list of key requirements,
 - delivery choices,
 - estimate of size and complexity,
 - project planning,
 - tool selection.

- Analysis
 - storyboard,
 - use cases,
 - prototype screens,
 - top-level design.
- Design
 - complete logical design,
 - name and document all structured links,
 - name and document all pages,
 - complete physical design,
 - name and document all physical directories and files,
 - document mapping logical to physical design,
 - naming conventions for physical design.
- Implementation
 - obtain or write all source material,
 - conversion to hypertext,
 - build in links and anchors,
 - document unstructured links,
 - use post-processor,
 - implement configuration management process,
 - transfer to live environment,
 - re-edit any absolute links that were specific to the development environment,
 - page test (each page),
 - system test,
 - customer acceptance test.
- Maintenance
 - establish responsibilities for live operation,
 - establish policy for monitoring remote links,
 - establish process for handling feedback,
 - establish logging and reporting process,
 - establish change control process,
 - plan for enhancement.

In practice, there are many diversions from this rather idealised picture (as there are from any of the software life cycles). Nonetheless, it does provide a baseline for control and establishes some boundaries within which the creative process can flourish. Once a reasonable life cycle is in place, attention can be focused on what is actually built — the design of the product itself.

6. The design process

The media engineering design process is split into two main activities:

- the logical design during which all the main information objects and the links between them are designed,
- the physical design in which these are mapped on to 'real' files and directory structures on a server.

Media engineering design follows the practice of progressive functional decomposition. Firstly, the complete system is considered and decisions are made regarding the links that are common to every page in the system. An example system-level design is illustrated in Fig 4.

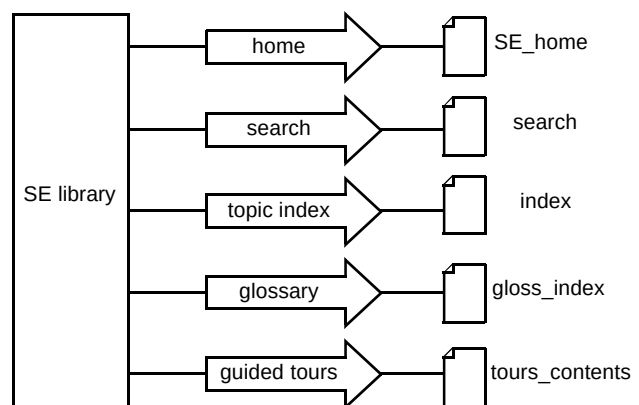


Fig 4 Example system-level design.

This example is drawn from the case study described later on. The system is an on-line library of software engineering material (the SE library). This design records the fact that every page of the system will have a link to the home page, search page, topic index, glossary and guided tours.

The next step would be to decompose the system into its main functional units before going on to design each one in detail.

In this example, it was decided that the system would consist of an 'Access pages' component, together with components for each of the technical topics. The access pages consist of 'Home', 'Search', 'Index', 'Glossary' and 'Tours' pages. The design for these is recorded in Fig 5.

The Home page actually has no links from it other than the generic links that are common to all pages.

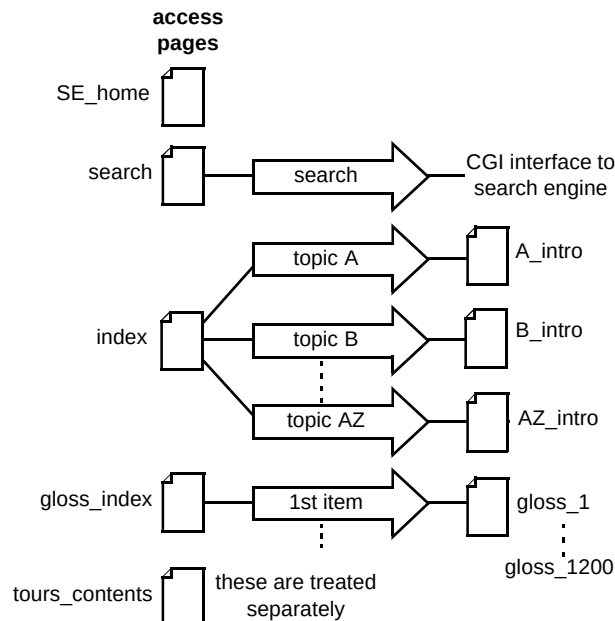


Fig 5 Link design for the Access pages.

The Search page links (via a standard interface) to an existing search engine. The Index page has links into the introductory pages of each of a set of topics, from Topic A, through to Topic AZ.

Likewise, the Glossary Index leads to one of the 1200 glossary definitions. The structure below the guided tours is somewhat more complex and is left at this stage (see West and Norris [2] for a full explanation of tour construction).

The process of decomposition is continued until all structured links have been documented.

7. Case study — the software engineer's library

The ideas of media engineering and the DIVA method that supports it were built up over a long period. One of the first real tests of the ideas, principles and techniques was the development, by the authors, of an Intranet-based reference manual.

Over a number of years, a great deal of manuals, guidelines, 'state-of-the-art reports' and similar documentation has been produced with the aim of promulgating consistent approaches and best-practice within BT's software engineering community of about 4000 software professionals. The initial brief for this job was to take the core of this information and turn it into an on-line software engineer's library. The resultant product was for internal consumption within the company rather than being accessible, for example, over the global Internet. Hence the end users were a relatively known community. One added complication that was thrown in at the outset, though, was that the structure should also be suitable for a variant to be produced as an external product.

Some of the particular challenges that this task presented were:

- the source information was in a multitude of different formats (at least four different word processors and three different graphics formats),
- the source information was structured as documents rather than as hypertext, some of the documents extending to hundreds of pages and therefore not able to be converted directly to hypertext 'pages',
- the resource available for the task was very limited — indeed this was largely done as a 'spare time' activity by the two present authors, aided by a willing vacation student.

Early on in the task, we set ourselves some basic principles and ground rules which we endeavoured to follow throughout the job. These were:

- we would apply the Diva method as rigidly as possible,
- we would use 'tours' as the principal way of structuring the material for the user — this meant that one of the basic principles of DIVA, polycontiguity, had to be implemented,
- we would document everything meticulously — this is an inherent part of the method but was seen as particularly important for this job because of its part-time nature and the need to be able to pick up where we left off at any time.

Our key success criteria from day one was to have a sufficiently attractive and useful product to engage a champion for its exploitation within our own organisation. A secondary (but still very important) measure was whether we could find an external agent to take ownership of a variant of the product.

The SE library was completed successfully within its initial ten week schedule. Our conclusion from experience of using the Diva method has been that it provides a good framework within which to work and contributed to a successful result in a short time-scale. Some of the key points made here are that:

- the emphasis on documenting the design and implementation at all stages is amply justified — with the final version containing over 1000 separate files, maintenance would have been all but impossible without it,
- the approach does nothing to stifle the creative side of page design — in fact, the notation provided a useful vehicle for communicating and building ideas on how the product should be constructed,

- once started, the method seems to work itself — much of DIVA is intuitive and reflects what designers do anyway,
- once clearly identified, some aspects of design can readily be reused, e.g. the concept of polycontiguity introduced earlier can be (and has been) implemented by constructing general-purpose navigation buttons (using Javascript) that can easily pick up the particular pages on the 'tour' to which they are related, hence one of our basic design principles being directly implementable with little effort,
- the process is not an unnecessary burden, on the contrary it contributes to on-time delivery — the full DIVA method incorporates good practice guidelines that help to avoid costly mistakes, and establish good practice in fixing requirements, with linking strategies and system, in using browser features such as frames and 'Cookies', in page design, with image maps (for example, the trade-offs between client-side and server-side maps), with colours in backgrounds and images, etc, all contributing to a better product in shorter time-scales.

The SE library exercised all aspects of the life cycle and design notation described earlier. Both have proved practical in designing this, and other, information products and stand fit for purpose. Just as with software engineering methods, there is plenty of scope to abuse DIVA and operational pressures will inevitably restrict the extent to which the method is followed. In view of this, the full method catalogues an extensive set of design guidelines [2], the intent being that some level of control is introduced, even when the full method is not followed.

8. Future directions

Media engineering is fast evolving and, as yet, far from established. It has been driven by a dramatic rise in the volume, complexity and importance of on-line information. To gauge how it might develop, this last section considers what the future might hold.

We start by taking stock of how we arrived where we now are. The evolution of information over the last twenty years is well illustrated by what has happened on the Internet. In the early days, there was the 'steam Internet'. This allowed one computer user to connect to a variety of remote computers and use their facilities. We could characterise this era as one where specialists used Telnet to log on to a networked system.

As the Internet became more sophisticated, people started to use software utilities such as Gopher to access the file index of a remote computer. We could say that we have moved into a second era, where the user connects to files.

The third era of information networking is familiar to many people — direct connection to remote information, most notably to pages on the World Wide Web. By now, the specialism of the user has been diluted to the extent that no knowledge of the details of the underlying computer network are necessary — the user connects to information [6]. The three eras are illustrated in Fig 6.

The transition between the three eras has been rapid — at least in terms of social evolution. So, connecting to information is very much a new phenomenon, one that has yet to fully develop. It seems likely, therefore, that media engineering will also have to develop as this era settles down. It should be clear, though, that (networked) hypermedia is here to stay.

It has been argued in this paper that media are defined more by their limitations than by their capabilities.

At different periods of man's history there have been a range of factors which limited the communication of information. Before the invention of the printing press, information was limited by the rate of output of the small number of skilled scribes — the cost of a single book was so great that very few could have access to them. Even in the case of printed media, a major limitation was until recently the sheer cost of the materials — indeed Penguin Books invented the paperback medium largely to bring books within the reach of working class people.

With the advent of modern electronic media, the cost of the medium is no longer the limitation. A compact disk (CD), for example, costs about 30p to manufacture and holds some 650 million characters; likewise, access to the Internet is free or at very low cost from many parts of the world. Admittedly, the equipment needed to extract the information from either CDs or the Internet (i.e. a PC) is beyond the means of all but the wealthiest classes on Earth, but certainly the trends in computing costs have continued to fall exponentially and these electronic media are ever coming within reach of more and more people. It seems that it will not be long before all human knowledge can be consigned to a piece of plastic the size of a credit card. The progress in this direction is illustrated in Fig 7.

So what is the current limitation on information access? Increasingly, it is the ability to search and 'navigate' the material and, ironically, all the other trends (those that make it easier to publish information as well as those that make it cheaper to store and distribute information) conspire to increase the complexity of the information and make it ever more difficult for the end user to navigate the 'information

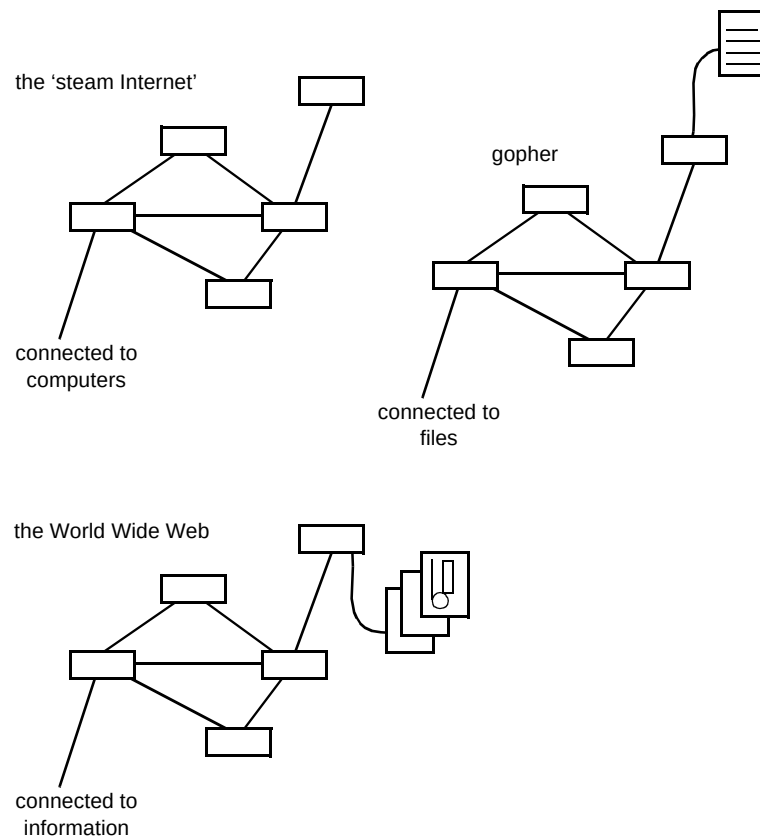


Fig 6 Three eras of information networks.

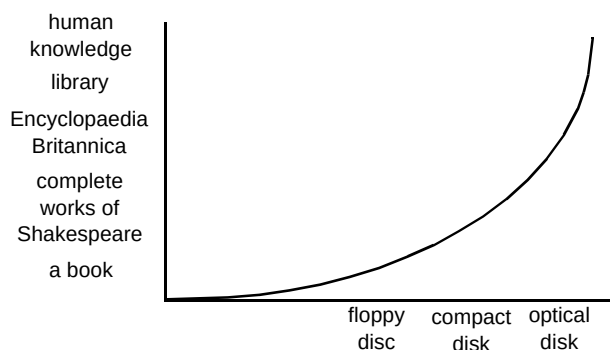


Fig 7 The ever-growing capacity of technology.

space'. CD technology allows us to put the contents of 1000 books in an object the size of a beer mat — it does nothing to assist in finding the required 'needle' in this highly compressed 'haystack'. The next generation of CDs will allow us to put 15 000 books (more than the average person reads in a lifetime) on a similar beer mat. The situation with the Internet is the same — the World Wide Web offers the end user at least 50 000 000 pages of information and very little discernible structure.

Webwatcher Gordon Bell's verdict is nicely summed up thus:

"I claim that we are all cats. Why? Well cats chase mice. And what is our life about? It's

mousing up and around these trees and then going up the trees and then down the trees by clicking on the browser's Back button. That's how we're spending our time."

It seems that our exploitation of this media will be limited by how well the information it carries is engineered. With such quantities available, a free format WWW would rapidly become a World Wide Waste of time.

9. Conclusions

Even before the dramatic growth of the World Wide Web, Herbert Simon observed that 'What information consumes is rather obvious: it consumes the attention of its recipients. Hence a wealth of information creates a poverty of attention and a need to allocate that attention efficiently among the over-abundance of information sources that might consume it'. The basic premise of this paper has been that there is a need to design information so that it can be readily consumed.

The method that has been developed for this purpose is very much driven by the idea that in the very near future, information dissemination will be limited by the user's ability to navigate. So the information provider will be limited by their ability to structure what they want to say — not by their access to technology nor by the value of their material. So, the usability (and ultimately, value) of any information provider's offerings will be determined by the

rigour with which they prepare their wares. A large part of our purpose in defining media engineering has been to lay a much needed framework for the construction of information products.

Acknowledgements

The authors are grateful to John Helleur and Gaynor Beale-Garland of BT, Professor David Bustard at the University of Ulster, Rosemary Seagrief of Websters-Encarta and Peter Croasdale of the BBC Multimedia unit for their advice and encouragement. Many of the ideas in this paper are taken from a book in the Wiley-BT series published by John Wiley and Sons and we would thank the publisher, Ann-Marie Halligan, for her support throughout. Finally, we thank Mark Pickford of Salford University for his hard work in building the SE library.

References

- 1 Lyons M and Gell M A: 'Companies and communications in the next century', *British Telecommunications Eng J*, 13, Part 2 (1994).
- 2 West S M and Norris M T: 'Media engineering: the design and build of information products', John Wiley & Sons (1997).
- 3 Jacobson I: 'Object oriented software engineering', Addison Wesley (1992).
- 4 Norris M T and Rigby P: 'Software engineering explained', John Wiley & Sons (1992).
- 5 Booch G: 'Object-oriented design with applications', Benjamin Cummings (1991).
- 6 Davies N J, Weeks R and Revett M C: 'Information agents for the World Wide Web', *BT Technol J*, 14, No 4, pp 105—114 (October 1996).



Steve West obtained an MA degree in Physics from Oxford University and, after working in education for several years, joined BT Laboratories in 1980. He has worked on numerous software and communications projects as well as contributing to the development of BT's Information Technology strategy.

Recently, he has been responsible for championing the move of internal technical standards documents (for the IT infrastructure and software engineering) on to World Wide Web servers on the BT intranet.

Currently he is part of the OSS design management team which is responsible for continued excellence in the design of BT's computing systems.



Mark Norris was awarded BSc and PhD degrees by the University of Glasgow in 1976 and 1979 respectively.

He has worked on numerous software and communications projects. Recently he has been responsible for systems to support data services.

He is a Fellow of the IEE and a regular contributor to the Encarta multimedia encyclopedia.